

PRACTICAL MACHINE LEARNING GUIDE

台灣產業機器學習模型實務研讀教材

從問題定義、模型選擇到部署監控的實作指南

實務導向	十種核心模型	完整建模流程
------	--------	--------

適用於表格資料、文字、分群、降維與深度學習專案

目錄

使用說明	8
1. 從商業問題轉成機器學習問題	8
1.1 先寫決策，不要先選模型	8
1.2 判斷任務型態	8
1.3 先建立非模型基準	8
2. 實務建模完整流程	9
2.1 定義資料時間線	9
2.2 建立資料字典與品質檢查	9
2.3 切分資料與建立 Pipeline	9
2.4 訓練、調參與最終測試	10
2.5 部署不是存一個模型檔	10
3. 模型評估：先選對指標	10
3.1 回歸指標	10
3.2 分類指標	10
3.3 分群與降維指標	10
3.4 門檻要依成本與容量設定	10
4. 實務模型選擇流程	11
4.1 第一層：依輸出選家族	11
4.2 第二層：依限制縮小範圍	11
4.3 第三層：用診斷結果決定下一步	11
4.4 最小可行比較組合	11
5. 線性回歸 Linear Regression	11
能解決什麼	11
實作重點	12

重要參數與選擇	12
何時不要用	12
台灣產業情境	12
實務檢查	12
6. 邏輯回歸 Logistic Regression	12
能解決什麼	12
實作重點	13
重要參數與選擇	13
何時不要用	13
台灣產業情境	13
實務檢查	13
7. 決策樹 Decision Tree	13
能解決什麼	13
實作重點	14
重要參數與選擇	14
何時不要用	14
台灣產業情境	14
實務檢查	14
8. 隨機森林 Random Forest	14
能解決什麼	14
實作重點	15
重要參數與選擇	15
何時不要用	15
台灣產業情境	15
實務檢查	15
9. 支持向量機 Support Vector Machine	15

能解決什麼	15
實作重點	16
重要參數與選擇	16
何時不要用	16
台灣產業情境	16
實務檢查	16
10. K-近鄰演算法 K-Nearest Neighbors	16
能解決什麼	16
實作重點	17
重要參數與選擇	17
何時不要用	17
台灣產業情境	17
實務檢查	17
11. 單純貝氏 Naive Bayes	17
能解決什麼	17
實作重點	17
重要參數與選擇	18
何時不要用	18
台灣產業情境	18
實務檢查	18
12. K-Means 分群	18
能解決什麼	18
實作重點	18
重要參數與選擇	19
何時不要用	19
台灣產業情境	19

實務檢查	19
13. 主成分分析 Principal Component Analysis	19
能解決什麼	19
實作重點	19
重要參數與選擇	19
何時不要用	20
台灣產業情境	20
實務檢查	20
14. 類神經網路與深度學習	20
能解決什麼	20
實作重點	20
重要參數與選擇	20
何時不要用	21
台灣產業情境	21
實務檢查	21
15. 常見失敗：過度擬合、洩漏與不平衡	21
15.1 過度擬合	21
15.2 資料洩漏	21
15.3 類別不平衡	21
15.4 標籤品質	22
16. 模型比較實戰：設備異常預警	22
16.1 問題定義	22
16.2 資料設計	22
16.3 最小比較	22
16.4 選擇與門檻	22
16.5 上線驗證	22

17. 從基準模型到部署	22
17.1 模型登錄內容	22
17.2 上線模式	23
17.3 監控四個層次	23
17.4 重訓與回滾	23
18. 模型選擇速查表	23
19. 建模前後檢查清單	24
建模前	24
訓練中	24
上線前	24
20. 建議學習路線	24
21. 特徵工程實務手冊	25
21.1 特徵必須對應可重複的資訊	25
21.2 數值特徵	25
21.3 類別特徵	25
21.4 時間特徵	25
21.5 文字與影像特徵	25
21.6 特徵選擇	26
22. 實驗設計與可重現交付	26
22.1 一次實驗只回答一個主要問題	26
22.2 交叉驗證不是固定五折	26
22.3 誤差分析模板	26
22.4 模型卡應寫什麼	26
22.5 驗收會議的決策資料	27
23. 問題排除手冊	27
23.1 模型幾乎沒有預測能力	27

23.2 訓練很好，驗證很差	27
23.3 離線很好，上線後警示暴增	27
23.4 模型分數正常，但業務沒有改善	27
23.5 特徵重要度與領域直覺衝突	28
23.6 模型需要頻繁重訓	28
23.7 團隊不知道該繼續調參還是停止	28

使用說明

這份教材不是模型百科，而是一份用來完成實際專案的操作指南。閱讀時請持續問三個問題：目前要預測或探索的是什麼、錯誤的代價是什麼、模型產生的結果要由誰採取行動。若這三題沒有答案，再精密的演算法也只會得到難以使用的分數。

教材沿用 Task 1 資訊圖表中的十個模型：線性回歸、邏輯回歸、決策樹、隨機森林、支持向量機、K-近鄰、單純貝氏、K-Means、主成分分析，以及類神經網路 / 深度學習。品牌與產業案例均作為「可行的應用情境」說明，不代表特定企業已採用文中描述的模型或流程。

建議先讀第 1 至 4 章，建立問題定義、資料切分、評估與選模觀念，再依工作需求閱讀個別模型。第一次實作時，不要同時比較所有演算法；先建立一個可解釋的基準模型，再針對已觀察到的瓶頸增加複雜度。

1. 從商業問題轉成機器學習問題

1.1 先寫決策，不要先選模型

一個可執行的機器學習題目，至少要寫清楚下列內容：

- 決策對象：誰會使用模型結果，例如品管工程師、授信人員、醫療團隊或電商營運人員。
- 決策時點：預測要提前幾分鐘、幾天或幾個月產生。可用時間會決定可取得的特徵。
- 預測單位：一片晶圓、一筆交易、一位病患、一道訂單，或一個會員。
- 目標變數：實際要預測的數值、類別或事件，以及標籤如何產生。
- 行動門檻：分數達到多少才要警示、人工覆核、阻擋或派工。
- 錯誤成本：漏掉真正異常與誤報正常資料，哪一種代價較高。
- 成功條件：除了模型指標，也要包含工時、成本、處理量、延遲或風險改善。

不完整的題目是「做一個客戶流失模型」。較好的題目是「每週一針對仍有效的訂閱戶，預測未來 30 天內取消的風險；營運團隊最多可聯繫 2,000 人，因此以 top-k recall、實際挽回率與每位成功挽回成本評估」。後者才足以決定資料時間窗、模型輸出、門檻與驗證方式。

1.2 判斷任務型態

若目標是連續數值，例如售價、需求量或耗電量，先視為回歸問題。若目標是離散標籤，例如違約 / 未違約、瑕疵類型或是否轉入加護病房，視為分類問題。若沒有標籤，希望找出自然群組，可使用分群。若欄位太多、共線性嚴重或希望視覺化，考慮降維。影像、語音、長文本或高度非線性的龐大資料，才優先考慮深度學習。

「預測明天是否缺貨」是分類；「預測明天缺貨幾件」是回歸；「找出門市補貨模式相近的群組」是分群。相同資料可以有不同任務，選擇取決於最後要做的決策。

1.3 先建立非模型基準

基準不一定是機器學習。需求預測可先用「沿用上周同日銷量」，分類可先用多數類別或既有規則，推薦可先用熱門商品。新模型若只在離線指標微幅勝出，卻增加大量維護成本，就不一定值得上線。基準模型的價值是提供最低可接受門檻，也讓團隊知道改善來自演算法、資料，還是評估方式改變。

2. 實務建模完整流程

2.1 定義資料時間線

每筆樣本都應畫出三個時間：觀察截止時間、預測產生時間、結果確認時間。特徵只能使用預測當下已知的資訊。若用月底才結算的欄位預測月中事件，或用事後人工處理結果作為特徵，就會發生資料洩漏。離線成績可能極高，上線後卻失效。

時間序列或會隨時間漂移的資料，應依時間先後切分訓練集、驗證集與測試集，不要隨機打散。若同一病患、設備或客戶有多筆紀錄，也要以群組切分，避免同一實體同時出現在訓練與測試資料中。

2.2 建立資料字典與品質檢查

資料字典至少包含欄位名稱、商業定義、單位、資料型態、允許範圍、缺失意義、更新頻率、來源系統與負責人。實作前先檢查：

- 主鍵是否唯一，資料粒度是否一致。
- 標籤比例及時間分布是否合理。
- 缺失值是未知、不適用、尚未發生，還是系統故障。
- 類別欄位是否有拼字、大小寫或編碼不一致。
- 數值是否有單位混用、極端值或不可能值。
- 訓練與上線環境能否取得相同欄位與轉換邏輯。

不要一律用平均值填補。缺失本身可能代表風險，也可能是流程改版造成。應先理解形成原因，再決定刪除、填補、增加缺失指標或改用能處理缺失的模型。

2.3 切分資料與建立 Pipeline

所有會從資料學習參數的處理，例如標準化、缺失值填補、類別編碼、特徵選擇與 PCA，都必須只在訓練資料上 fit，再套用到驗證與測試資料。使用 scikit-learn 的 Pipeline 與 ColumnTransformer，可以減少手動流程不一致和交叉驗證洩漏。

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="median")),
    ("scale", StandardScaler()),
])
category_pipe = Pipeline([
    ("impute", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore")),
])
preprocess = ColumnTransformer([
    ("num", numeric_pipe, numeric_columns),
    ("cat", category_pipe, category_columns),
])
```

2.4 訓練、調參與最終測試

訓練集用來學習模型，驗證集或交叉驗證用來選特徵、模型與超參數，測試集只在方案定案後使用一次。若反覆查看測試結果再修改模型，測試集就已經變成驗證集，最終成績會過度樂觀。

調參不應只追逐單一平均分數。要同時觀察各折變異、不同時間區段、重要族群、推論速度、模型大小與機率校準。若兩個模型表現接近，通常優先選擇較容易解釋、部署與監控的方案。

2.5 部署不是存一個模型檔

可部署成果至少包含：資料前處理、模型權重、版本資訊、輸入 schema、輸出定義、閾值、例外處理、效能基準與監控規則。批次模型需定義排程、重跑與資料遲到策略；即時模型需定義延遲、逾時、降級與容量。模型無法回應時，系統要回到既有規則、人工流程或安全預設，而不是讓整個決策停止。

3. 模型評估：先選對指標

3.1 回歸指標

MAE 是平均絕對誤差，單位與目標相同，容易向業務解釋。RMSE 對大誤差懲罰較重，適合大偏差特別昂貴的情境。MAPE 以百分比呈現，但目標接近零時會失真，也會不對稱地處罰高估與低估。 R^2 表示相對於只預測平均值的改善程度，不能單獨代表誤差是否可接受。

實務上應報告整體 MAE、關鍵區間 MAE、誤差分布與最差案例。例如房價估值模型的整體 MAE 可接受，但高總價物件誤差可能特別大；只看平均值會掩蓋真正風險。

3.2 分類指標

Accuracy 只有在類別接近平衡、各種錯誤成本相近時才適用。Precision 表示被模型判定為正類的樣本中有多少是真的，適合人工覆核容量有限、誤報成本高的情境。Recall 表示真正正類有多少被找出，適合漏報代價高的異常、疾病或詐欺偵測。F1 是 precision 與 recall 的折衷，但仍未反映實際成本。

ROC-AUC 衡量整體排序能力，在正類極少時可能顯得過度樂觀；此時應優先查看 PR-AUC、precision@k、recall@k。若輸出的機率會直接用於風險定價或資源分配，還要檢查 calibration curve、Brier score，必要時做機率校準。

3.3 分群與降維指標

分群沒有唯一正確答案。Silhouette score、Davies-Bouldin index 可協助比較群內緊密與群間分離，但更重要的是群組是否穩定、可描述、規模合理，且能導向不同策略。若每次重跑群組就大幅改變，或營運無法對各群採取不同動作，即使數學指標漂亮也沒有價值。

PCA 可用累積解釋變異比例決定保留成分數，但不能只追求 95%。應比較降維後下游模型表現、訓練速度與解釋性。低變異方向仍可能包含少數但重要的異常訊號。

3.4 門檻要依成本與容量設定

分類模型先產生分數，再由門檻轉成行動。0.5 不是自然法則。若團隊每天只能覆核 100 筆，可從最高風險排序取前 100 筆；若漏掉一次設備故障的代價極高，則可能降低門檻以提高 recall。門檻應在驗證資料上決定，並用金額、工時、處理量和風險表達，而非只用 F1。

4. 實務模型選擇流程

4.1 第一層：依輸出選家族

1. 目標是連續數值：先做線性回歸，若殘差顯示明顯非線性，再比較樹模型或神經網路。
2. 目標是類別：先做邏輯回歸；需要可讀規則可比較決策樹，需要較強表格資料表現可比較隨機森林。
3. 樣本中等、特徵高維且邊界可能清楚：比較 SVM。
4. 希望以相似案例直接判斷、資料量不大：比較 KNN。
5. 文字詞頻等高維稀疏資料，需要很快的基準：比較單純貝氏。
6. 沒有標籤，要探索自然群組：使用 K-Means，先確認尺度與群組假設。
7. 特徵過多或高度相關：先評估 PCA 是否能壓縮，再比較下游模型。
8. 影像、語音、長文本或大量複雜非線性資料：在資料量、算力和維護能力足夠時採用深度學習。

4.2 第二層：依限制縮小範圍

- 必須清楚解釋係數與方向：線性回歸、邏輯回歸。
- 必須提供人能閱讀的規則：淺層決策樹。
- 表格資料且希望穩健處理非線性：隨機森林。
- 特徵數高於樣本數：線性 SVM、單純貝氏或先做正則化 / 降維。
- 推論必須非常快：線性模型、單純貝氏、淺樹通常較容易控制。
- 訓練資料很少：優先簡單模型、領域特徵與交叉驗證，不要直接建立大型神經網路。
- 需要自然機率：邏輯回歸通常比 SVM、樹投票更容易校準，但仍要驗證。
- 邊緣裝置資源有限：評估模型大小、記憶體與延遲，不只比較準確率。

4.3 第三層：用診斷結果決定下一步

訓練與驗證都差，通常是欠擬合、特徵不足、標籤噪聲或問題不可預測；增加模型複雜度只是其中一種可能。訓練很好但驗證差，通常是過擬合、資料量不足、洩漏式特徵或分布不一致。離線很好但上線差，優先檢查特徵計算差異、資料延遲、族群改變與門檻，而不是立即換模型。

4.4 最小可行比較組合

表格分類專案可先比較邏輯回歸、淺層決策樹與隨機森林；表格回歸先比較平均值基準、線性回歸與隨機森林回歸；文字分類先比較 TF-IDF 加單純貝氏或線性模型，再判斷是否需要深度學習。這種小型比較組合能快速回答「非線性是否重要」「複雜模型是否值得」兩個核心問題。

5. 線性回歸 Linear Regression

能解決什麼

線性回歸預測連續數值，並假設目標可由特徵的加權總和加上誤差表示。它適合建立第一個基準、估計方向與幅度，以及需要清楚說明變數影響的情境。房價、需求、交期、成本與能源使用量都可先用它測試。

實作重點

線性關係不是要求每個欄位與目標看起來完全是直線，而是模型對參數呈線性。類別可經 one-hot 編碼，非線性可加入對數、平方或交互項，但每增加轉換都要能在上線時一致計算。特徵尺度不影響普通最小平方方法的預測能力，卻會影響係數比較與正則化。

主要假設包含殘差平均接近零、變異大致穩定、觀測值合理獨立，以及特徵沒有嚴重共線性。違反假設不代表模型不能預測，但會影響係數解釋與信賴區間。應畫殘差對預測值、誤差分布及時間趨勢，不要只看 R^2 。

重要參數與選擇

普通 LinearRegression 幾乎沒有核心超參數。當特徵多或共線性高時，使用 Ridge 的 L2 正則化讓係數較穩定；希望部分係數變成零可比較 Lasso，但其特徵選擇在高度相關欄位間可能不穩定。正則化強度應在交叉驗證中選擇，且數值欄位先標準化。

何時不要用

若殘差呈現明顯曲線、不同區間誤差差異巨大、重要交互作用難以人工列舉，或業務只關心極端尾部，單純線性回歸可能不足。不要因係數為正就宣稱因果；模型只描述控制其他已納入特徵後的統計關聯。

台灣產業情境

房地產估價可用坪數、屋齡、樓層、行政區與交通距離建立透明基準；製造業可估計製程參數與耗能或週期時間的關係。實務上應依地區、產品族群或時間檢查誤差，避免整體平均掩蓋特定區域失準。

```
from sklearn.linear_model import Ridge
from sklearn.pipeline import Pipeline

model = Pipeline([
    ("preprocess", preprocess),
    ("model", Ridge(alpha=1.0)),
])
model.fit(X_train, y_train)
pred = model.predict(X_valid)
```

實務檢查

- 與只預測平均值或上期數值的基準比較。
- 檢查 MAE、RMSE、殘差圖與各重要族群誤差。
- 確認係數方向是否符合資料定義，而非只符合直覺。
- 對極端值做資料查核，不要直接全部刪除。

6. 邏輯回歸 Logistic Regression

能解決什麼

邏輯回歸用特徵的線性組合估計事件發生機率，主要用於二元分類，也可延伸多分類。它是風險評分、流失、違約、疾病與品質判定的強力基準。係數可轉成勝算比，適合需要說明影響方向的場景。

實作重點

模型假設特徵與 log-odds 近似線性，而不是特徵與機率線性。連續欄位若呈現非線性，可做分箱、樣條或交互作用。類別欄位需編碼，數值通常建議標準化，以利正則化和收斂。

類別不平衡時，不要只看 accuracy。先確認正類定義，再比較 PR-AUC、recall、precision、top-k 表現及混淆矩陣。class_weight 可以改變學習時對少數類別的重視程度，但不會自動產生符合真實比例的機率，使用後要重新檢查校準。

重要參數與選擇

C 是正則化強度的倒數，C 越小代表限制越強。penalty 可選 L1、L2 或 elastic net；solver 必須與 penalty 相容。max_iter 不足常造成未收斂警告，應先檢查尺度、共線性和資料品質，而不是只把迭代數設得很大。

何時不要用

若類別邊界高度非線性、特徵交互作用很多且難以事先設計，樹模型可能更合適。若資料量很小而特徵極多，必須加強正則化與交叉驗證。係數可解釋不等於流程公平，敏感族群仍需分組評估。

台灣產業情境

金融風險可用於違約或人工覆核排序；醫療可估計特定事件風險；製造可判斷批次是否需要加驗。模型輸出應連到實際容量，例如每天覆核前 200 筆，而非固定用 0.5 門檻。

```
from sklearn.linear_model import LogisticRegression

clf = Pipeline([
    ("preprocess", preprocess),
    ("model", LogisticRegression(
        C=1.0, penalty="l2", class_weight="balanced", max_iter=1000
    )),
])
clf.fit(X_train, y_train)
score = clf.predict_proba(X_valid)[:, 1]
```

實務檢查

- 先畫 precision-recall curve，再依成本與容量選門檻。
- 比較不同月份、產品與客群的 recall 和誤報率。
- 檢查機率校準，不把未校準分數直接當真實風險。
- 記錄特徵編碼與參考類別，避免係數解讀錯誤。

7. 決策樹 Decision Tree

能解決什麼

決策樹反覆選擇特徵與切點，把資料分成較純的節點。它能處理非線性與交互作用，並以 if-then 規則呈現。當需求是形成可閱讀的判斷流程、找出初步分群規則或向領域專家討論時，淺層樹特別有用。

實作重點

未限制的樹很容易記住訓練資料。max_depth、min_samples_leaf、min_samples_split 與 cost-complexity pruning 是主要控制工具。實務上寧可使用稍淺、每個葉節點樣本較多的樹，換取規則穩定性。

樹不需要標準化，但前處理仍要處理缺失與類別編碼。一般 one-hot 編碼會讓同一類別欄位拆成多個判斷，解讀時要回到原始欄位。feature_importances_ 是不純度下降的彙總，會偏好可切分點多的特徵，不能當因果證據；可搭配 permutation importance。

重要參數與選擇

criterion 控制分裂品質；分類常見 gini 或 entropy，回歸常用 squared_error 或 absolute_error。max_depth 直接限制規則層數，min_samples_leaf 可避免只服務少數樣本的葉節點。class_weight 可處理不平衡，但仍需依真實成本選門檻。

何時不要用

單棵樹對資料小變動敏感，預測通常不如集成樹穩定。若主要目標是最高預測效果，可比較隨機森林；若需要平滑機率或線性方向，邏輯回歸更自然。規則太深時，所謂可解釋性會快速消失。

台灣產業情境

製造品管可用淺樹整理「哪些製程條件組合需要加驗」的候選規則；客服可用於路由與優先級。正式上線前應由領域人員確認規則是否使用了事後欄位、代理敏感資訊或偶然資料偏差。

```
from sklearn.tree import DecisionTreeClassifier, export_text

tree = Pipeline([
    ("preprocess", preprocess),
    ("model", DecisionTreeClassifier(
        max_depth=4, min_samples_leaf=50, class_weight="balanced",
        random_state=42
    )),
])
tree.fit(X_train, y_train)
```

實務檢查

- 比較訓練與驗證成績，深度增加但驗證不升即停止。
- 確認每個葉節點的樣本量與正類比例。
- 用不同時間切分重訓，檢查主要規則是否穩定。
- 將規則視為決策支援，不直接推論因果。

8. 隨機森林 Random Forest

能解決什麼

隨機森林利用 bootstrap 抽樣訓練多棵樹，並在每次分裂只考慮部分特徵，最後以投票或平均得到結果。它能捕捉表格資料中的非線性與交互作用，通常是可靠、調參負擔相對低的強基準。

實作重點

`n_estimators` 增加通常會讓結果更穩定，但訓練、記憶體與推論成本也增加。`max_depth`、`min_samples_leaf` 和 `max_features` 控制單棵樹複雜度與樹之間差異。樹高度相關時，增加數量的收益有限。

隨機森林不要求標準化，但類別編碼、缺失處理與時間切分仍不可省略。`out-of-bag score` 可提供額外估計，但不能取代符合業務時間線的驗證。預測機率來自樹投票比例，不一定校準良好。

重要參數與選擇

先固定合理的 `n_estimators`，例如 300 至 500，再調 `max_depth`、`min_samples_leaf`、`max_features`。若資料極度不平衡，可比較 `class_weight`、`balanced subsampling` 或依業務容量選門檻。不要用完整網格窮舉所有組合；先從會控制過擬合與成本的參數開始。

何時不要用

需要逐一說明係數方向或部署環境資源很有限時，隨機森林可能不是首選。高維稀疏文字資料通常線性模型或單純貝氏更有效率。當資料規模很大且追求極低延遲時，要實測模型大小與推論吞吐量。

台灣產業情境

可用於半導體與電子製造的多參數品質風險、醫療風險排序、房價非線性估值或零售需求預測。模型指出的重要特徵應用 `permutation importance` 或 `SHAP` 等方法再確認，並檢查不同批次與產品族群是否一致。

```
from sklearn.ensemble import RandomForestClassifier

forest = Pipeline([
    ("preprocess", preprocess),
    ("model", RandomForestClassifier(
        n_estimators=400, max_depth=12, min_samples_leaf=20,
        max_features="sqrt", class_weight="balanced_subsample",
        n_jobs=-1, random_state=42
    )),
])
forest.fit(X_train, y_train)
```

實務檢查

- 與邏輯回歸比較，確認非線性帶來的提升是否值得。
- 記錄模型檔大小、單筆與批次推論時間。
- 使用 `permutation importance`，避免只依賴樹內建重要度。
- 檢查機率校準與門檻，不直接使用預設分類結果。

9. 支持向量機 Support Vector Machine

能解決什麼

SVM 尋找能最大化類別間隔的決策邊界。透過 `kernel`，可在不明確建立高維特徵的情況下形成非線性邊界。它常適用於樣本量中等、特徵維度高、邊界相對清楚的分類問題。

實作重點

SVM 對尺度非常敏感，數值特徵必須在 Pipeline 中標準化。線性 SVM 適合高維稀疏資料；RBF kernel 可處理非線性，但訓練成本會隨樣本數快速增加。資料量很大時，應先比較 LinearSVC、SGDClassifier 或其他線性模型。

C 控制錯誤懲罰與間隔寬度；C 大會更努力分類訓練資料，可能過擬合。RBF 的 gamma 控制單一樣本影響範圍；gamma 太大會形成非常彎曲的邊界。兩者要一起用對數尺度搜尋。

重要參數與選擇

先用線性 kernel 建立基準，再確認 RBF 是否有穩定提升。probability=True 會增加訓練成本，因為需要額外校準；若只需要排序與門檻，可先使用 decision_function，再用獨立驗證資料做校準。

何時不要用

數十萬以上樣本、頻繁線上重訓或要求直接規則解釋時，kernel SVM 可能不合適。大量缺失或未處理類別資料也不適合直接輸入。不要因高維就自動使用 SVM，仍要和正則化線性模型比較。

台灣產業情境

可用於 IC 測試訊號、設備頻譜、文本或基因等高維資料的分類。若每次量測的尺度會因設備校正改變，標準化參數必須隨模型版本保存，並監控輸入分布。

```
from sklearn.svm import SVC

svm = Pipeline([
    ("preprocess", preprocess),
    ("model", SVC(
        kernel="rbf", C=10, gamma="scale",
        class_weight="balanced", probability=False
    )),
])
svm.fit(X_train, y_train)
decision_score = svm.decision_function(X_valid)
```

實務檢查

- 將標準化包含在 Pipeline，避免交叉驗證洩漏。
- 同時比較線性與 RBF，不直接從複雜 kernel 開始。
- 測量訓練時間、支援向量數量與推論延遲。
- 若要輸出機率，使用獨立資料校準並驗證穩定性。

10. K-近鄰演算法 K-Nearest Neighbors

能解決什麼

KNN 不先學出明確函數，而是在預測時尋找最近的 K 個訓練樣本，以投票或平均得到結果。它直觀、容易形成相似案例解釋，也能作為局部模式基準。

實作重點

距離決定一切，因此數值特徵要標準化，類別與數值混合時要慎選編碼與距離。無關特徵會扭曲鄰近關係，高維空間中的距離也會逐漸失去區別，這是維度災難。可先做特徵選擇或 PCA，但要在交叉驗證 Pipeline 內完成。

K 太小容易受雜訊影響，K 太大會把局部差異平均掉。weights="distance" 可讓較近樣本權重較高。距離度量要符合資料意義；一般歐氏距離不適合所有欄位。

重要參數與選擇

調整 n_neighbors、weights 與 metric。使用交叉驗證時應觀察不同 K 的穩定區間，不要只選偶然最高的一點。資料量增大後，預測需要搜尋大量樣本；可評估近似最近鄰或改用會先學習參數的模型。

何時不要用

高維、特徵尺度混亂、訓練集巨大或即時延遲嚴格時，KNN 往往不理想。若資料分布隨時間改變，舊樣本會持續參與鄰近判斷，必須設計時間窗或更新策略。

台灣產業情境

零售可尋找購買輪廓相似的會員或商品，維修可找歷史訊號相近的故障案例。使用相似案例時要避免把敏感資訊或結果欄位納入距離，也要向使用者顯示真正有意義的相似特徵。

```
from sklearn.neighbors import KNeighborsClassifier

knn = Pipeline([
    ("preprocess", preprocess),
    ("model", KNeighborsClassifier(
        n_neighbors=15, weights="distance", metric="minkowski", p=2
    )),
])
knn.fit(X_train, y_train)
```

實務檢查

- 檢查標準化與距離是否符合欄位意義。
- 移除無關欄位，觀察鄰居是否變得更合理。
- 測量資料量成長後的記憶體與單筆推論時間。
- 對展示出的相似案例做隱私與權限控制。

11. 單純貝氏 Naive Bayes

能解決什麼

單純貝氏利用貝氏定理，並假設給定類別後各特徵條件獨立。這個假設常不完全成立，但在高維稀疏文字分類中仍能得到快速、有效的基準。客服信件分流、垃圾訊息與文件主題分類是典型用途。

實作重點

MultinomialNB 適合詞頻或非負計數，BernoulliNB 適合是否出現的二元特徵，GaussianNB 適合近似常態的連續特徵。模型種類必須與資料表示相容。TF-IDF 通常可與 MultinomialNB 搭配，但要實際驗證。

α 是平滑參數，可避免未在某類別出現的詞得到零機率。 α 太小可能過度相信稀有詞，太大會把差異沖淡。文字前處理不要過早刪除否定詞、產品代碼或領域縮寫，這些可能正是分類關鍵。

重要參數與選擇

先建立簡單的字詞 n-gram 與 α 搜尋，再分析混淆案例。類別先驗可用資料估計，也可在業務理由充分時指定。若類別比例隨時間明顯改變，先驗與門檻要監控。

何時不要用

特徵間依賴非常重要、需要複雜語境理解或輸入不是適合的分布時，單純貝氏可能落後。它輸出的機率常過度自信，不應未經校準就當成真實風險。

台灣產業情境

電商與客服中心可將來信分到退貨、付款、物流或帳號問題；製造異常報告可做初步類別分流。模型應保留「無法確定 / 轉人工」機制，並監控新產品名稱與新型問題造成的詞彙漂移。

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB

text_clf = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=3)),
    ("model", MultinomialNB(alpha=0.5)),
])
text_clf.fit(text_train, y_train)
```

實務檢查

- 依資料型態選 Multinomial、Bernoulli 或 Gaussian 版本。
- 查看各類別常見詞，排除資料洩漏與流程代碼。
- 建立低信心轉人工的門檻。
- 定期檢查新詞、未知詞與類別比例變化。

12. K-Means 分群

能解決什麼

K-Means 將樣本分配到最近的質心，反覆更新質心以最小化群內平方距離。它適合探索大致呈球狀、尺度可比較的群組，例如會員消費輪廓、門市型態或設備運轉模式。

實作重點

K-Means 對尺度、離群值與初始質心敏感。金額、次數與天數若直接混用，數值範圍最大的欄位會主導距離。通常需標準化，偏態金額可先做 log 轉換。離群值可能把質心拉離主要群體，應先分析而非機械刪除。

K 不是由 elbow 圖自動決定。應同時評估 inertia、silhouette、不同初始化的穩定度、群組規模、可描述性與可行性。若選出的群組只有統計差異，卻無法設計不同策略，分群沒有完成商業任務。

重要參數與選擇

`n_clusters` 決定群數；`init="k-means++"` 通常比隨機初始化穩定；`n_init` 應足以比較多次初始點。
`random_state` 用於重現，但仍要透過不同種子檢查穩定性。

何時不要用

群組形狀細長、密度不同、含大量類別欄位或有明顯離群點時，K-Means 假設可能不成立。分群不是分類，沒有標籤時不能宣稱某群「一定會流失」；需用後續資料驗證群組差異。

台灣產業情境

零售可依 RFM、品類與通路行為建立會員群；連鎖門市可依時段、客單與商品組合分型；製造可找設備運轉模式。每群應產生可讀名稱、代表樣本、主要特徵與可執行策略。

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

cluster_pipe = Pipeline([
    ("scale", StandardScaler()),
    ("model", KMeans(n_clusters=5, n_init=20, random_state=42)),
])
labels = cluster_pipe.fit_predict(X_cluster)
```

實務檢查

- 先確認每個特徵的尺度、偏態與業務意義。
- 比較多個 K、多個種子與不同抽樣的群組穩定度。
- 檢查是否出現過小或只由離群值構成的群。
- 為每群定義實際策略，並用後續成效驗證。

13. 主成分分析 Principal Component Analysis

能解決什麼

PCA 尋找資料變異最大的正交方向，將原始相關特徵轉成較少的主成分。它可用於壓縮、去除共線性、視覺化、加速下游模型或建立製程綜合指標。

實作重點

PCA 對尺度敏感，若欄位單位不同通常先標準化。主成分是原始特徵的線性組合，符號方向可整體翻轉，不影響數學結果。解釋時應看 loading，而不是把 PC1 當成天然存在的業務欄位。

解釋變異高不代表對預測目標重要，因為 PCA 不使用標籤。少數類別訊號可能位於低變異方向。若目的是提高分類或回歸效果，成分數必須用下游模型的交叉驗證決定。

重要參數與選擇

`n_components` 可指定成分數或累積解釋變異比例。`svd_solver` 依資料形狀與規模選擇。高維稀疏文字資料通常使用 TruncatedSVD，因為標準 PCA 的中心化會破壞稀疏性。

何時不要用

若每個原始特徵都必須直接解釋、資料關係高度非線性，或低變異特徵可能是關鍵告警，PCA 未必適合。不要在全資料上先做 PCA 再切分；這會讓測試資料參與成分學習。

台灣產業情境

半導體封裝或設備監控可將大量相關感測器壓縮成少數綜合方向，供趨勢監控或後續分類；醫療與生物資料可用於視覺化樣本結構。上線後需監控 loading 適用性與成分分布是否漂移。

```
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

pca_clf = Pipeline([
    ("preprocess", preprocess),
    ("pca", PCA(n_components=0.95, random_state=42)),
    ("model", LogisticRegression(max_iter=1000)),
])
pca_clf.fit(X_train, y_train)
```

實務檢查

- 將標準化與 PCA 放入交叉驗證 Pipeline。
- 同時比較壓縮前後的指標、速度與可解釋性。
- 檢查 loading，確認成分不是由錯誤單位或單一欄位支配。
- 不用累積解釋變異作為唯一成分數選擇標準。

14. 類神經網路與深度學習

能解決什麼

類神經網路透過多層線性轉換與非線性激活函數學習複雜表示。深度學習在影像、語音、長文本、大量時序及高度複雜模式上特別有價值，也可用於龐大資料的表格問題，但不應因流行而取代簡單基準。

實作重點

先建立資料與簡單模型基準，再決定網路架構。訓練集、驗證集與測試集必須依實際時間或實體切分。數值特徵要標準化，類別可 embedding，影像需一致的尺寸、正規化與增強。所有前處理都要版本化。

過度擬合可透過資料增加、資料增強、weight decay、dropout、early stopping 與較小模型控制。欠擬合則可能來自表示不足、學習率不合、訓練時間不足或標籤本身不穩定。loss 下降不代表業務指標提升，應同時監控驗證指標。

重要參數與選擇

學習率通常比層數更先影響訓練成敗；batch size 影響記憶體、梯度噪聲與吞吐量；網路寬度與深度控制容量。應使用學習曲線、early stopping 與固定種子，並保存最佳驗證 checkpoint。遷移學習通常比從零訓練更適合資料有限的影像或文本專案。

何時不要用

資料只有數千筆表格紀錄、必須明確解釋每項因素、硬體與維運資源不足，或簡單模型已達成需求時，不應優先使用深度學習。模型越大，資料品質、版本管理、資安、成本與監控責任也越大。

台灣產業情境

智慧製造可用影像模型做外觀瑕疵候選框選，醫療可做影像輔助判讀，客服可做文本分類與摘要，設備可處理多變量時序。高風險領域應保留人工覆核、信心門檻、未知類別與稽核紀錄。

```
from sklearn.neural_network import MLPClassifier

mlp = Pipeline([
    ("preprocess", preprocess),
    ("model", MLPClassifier(
        hidden_layer_sizes=(128, 64), activation="relu",
        alpha=1e-4, learning_rate_init=1e-3,
        early_stopping=True, max_iter=300, random_state=42
    )),
])
mlp.fit(X_train, y_train)
```

實務檢查

- 先證明簡單模型不足，再增加網路複雜度。
- 保存資料版本、前處理、權重、設定與評估結果。
- 監控 loss、主要指標、族群差異、延遲與硬體用量。
- 對低信心、分布外輸入與模型故障設計安全降級。

15. 常見失敗：過度擬合、洩漏與不平衡

15.1 過度擬合

過度擬合的典型訊號是訓練成績持續改善，驗證成績停滯或下降。處理順序應是先確認切分與洩漏，再增加資料或改善標籤，接著降低特徵與模型複雜度、加強正則化，最後才考慮更複雜的集成。若資料分布已改變，正則化無法解決根本問題。

15.2 資料洩漏

常見洩漏包括：使用事件發生後才產生的欄位、先在全資料做標準化或 PCA、同一實體跨越訓練與測試、用人工處理結果預測是否需要人工處理、隨機切分帶有時間趨勢的資料。最有效的方法是建立欄位可用時間表，並對每項特徵詢問「預測當下真的拿得到嗎」。

15.3 類別不平衡

少數類別不代表一定要過採樣。先選對指標與門檻，再比較 class weight、欠採樣、過採樣或異常偵測方法。任何重新抽樣都只能在訓練折內進行。測試資料應保留真實比例，否則 precision 與實際工作量會失真。

15.4 標籤品質

標籤可能來自人工判定、後續流程、感測器規則或帳務結果。應抽樣檢查正負類，計算不同標註者一致性，記錄規則版本。若標籤本身經常矛盾，提升模型複雜度的收益有限。對爭議樣本可建立不確定類別或二次覆核流程。

16. 模型比較實戰：設備異常預警

16.1 問題定義

目標是在設備可能停機前 24 小時產生風險排序，維修團隊每天最多檢查 20 台。預測單位是「設備 - 日期」，正類是未來 24 小時內發生需停機處理的故障。主要指標不是 accuracy，而是 recall@20、precision@20、提前時間與每次有效檢查成本。

16.2 資料設計

特徵只使用預測時間前已完成的感測統計、近期告警次數、保養間隔、產品批次與環境資訊。故障後的維修代碼、零件更換與停機原因全部排除。相同設備的連續紀錄依時間切分：較早期間訓練，中間期間調參，最新期間測試。

16.3 最小比較

第一個基準是依近期告警次數排序。第二個模型是邏輯回歸，用來確認線性風險與可解釋性。第三個是淺層決策樹，協助工程師檢視規則。第四個是隨機森林，測試非線性和交互作用。若感測器欄位高度相關，再將標準化加 PCA 作為另一條 Pipeline，而不是先處理全資料。

16.4 選擇與門檻

假設隨機森林的 PR-AUC 最高，但邏輯回歸在 top 20 的 recall 幾乎相同，且推論與解釋成本較低，則可先部署邏輯回歸。每天只輸出最高風險 20 台，並附上主要風險因素、資料完整度與近期趨勢。門檻不是固定 0.5，而是依每日容量與最低風險要求共同決定。

16.5 上線驗證

先以 shadow mode 執行，不改變維修決策，只比較模型排序、工程師判斷與實際故障。確認資料延遲、特徵一致性和警示量後，再逐步讓模型影響部分設備。正式成效要比較停機時間、有效檢查比例、額外工時與漏報事故，而非只報告離線 AUC。

17. 從基準模型到部署

17.1 模型登錄內容

每個版本應記錄：模型名稱與版本、程式碼 commit、訓練資料時間範圍、特徵清單、前處理版本、超參數、驗證結果、門檻、適用族群、已知限制與負責人。只有一個 model.pkl 無法支援稽核與重現。

17.2 上線模式

批次推論適合每日或每週名單、需求預測與報表，系統簡單且容易重跑。即時 API 適合交易阻擋、即時推薦或線上判斷，但必須處理逾時、容量與特徵新鮮度。串流推論適合感測器與連續事件，還需處理順序、延遲與重複資料。先選最簡單且能滿足決策時效的模式。

17.3 監控四個層次

1. 系統：成功率、延遲、吞吐量、資源與錯誤碼。
2. 資料：缺失率、範圍、類別新值、特徵分布與資料延遲。
3. 模型：分數分布、警示量、校準、分族群指標與實際標籤回補後的效果。
4. 業務：成本、工時、轉換、停機、風險損失或其他最終成果。

模型指標通常需要等待標籤，因此資料與分數監控是早期預警。漂移只代表分布改變，不一定代表模型失效；仍要結合結果標籤與業務流程判斷是否重訓。

17.4 重訓與回滾

不要只設定「每月自動重訓」而沒有驗收。新模型應在固定測試窗與關鍵族群上通過最低門檻，再進行 shadow、canary 或 A/B 驗證。保留前一版本、前處理與門檻，確保可快速回滾。若標籤延遲很久，可先以資料品質和人工抽查作為中間安全閘門。

18. 模型選擇速查表

情境	優先基準	下一個比較	主要風險
連續數值且需解釋	線性 / Ridge	隨機森林回歸	非線性、異質變異
二分類且需機率	邏輯回歸	隨機森林	不平衡、校準、門檻
需要可讀規則	淺層決策樹	隨機森林	規則不穩、過擬合
表格資料非線性	隨機森林	神經網路或其他集成	推論成本、解釋性
中型高維分類	線性模型	SVM	尺度、訓練成本
相似案例判斷	KNN	線性或樹模型	維度災難、延遲
稀疏文字分類	單純貝氏	線性模型 / 深度學習	語境不足、詞彙漂移
無標籤客群探索	K-Means	其他分群方法	尺度、離群、不可行動
特徵高度相關	Ridge	PCA 加下游模型	解釋性、低變異訊號
影像、語音、長文本	遷移學習	客製深度模型	資料量、算力、監控

19. 建模前後檢查清單

建模前

- 已寫清楚決策者、決策時點、預測單位與行動。
- 標籤定義可重現，且抽樣檢查過品質。
- 每項特徵在預測當下確實可取得。
- 資料切分符合時間、群組與實際上線方式。
- 已建立簡單規則或統計基準。
- 指標能反映錯誤成本、容量與關鍵族群。

訓練中

- 所有學習式前處理都放入 Pipeline。
- 比較訓練與驗證結果，觀察變異而非只看平均。
- 超參數搜尋範圍有理由，不盲目窮舉。
- 已保存資料、程式、設定、模型與隨機種子版本。
- 已分析失敗案例、極端案例及分族群表現。

上線前

- 測試資料只在方案定案後使用。
- 輸入 schema、缺失、新類別與異常值有處理方式。
- 門檻依成本與容量決定，並有人工覆核或安全降級。
- 已測量推論延遲、批次時間、記憶體與模型大小。
- 已定義監控、標籤回補、重訓、驗收與回滾責任。
- 文件清楚描述適用範圍與不應使用的情境。

20. 建議學習路線

第一階段先熟悉資料切分、Pipeline、線性回歸與邏輯回歸，能獨立完成基準、指標與誤差分析。第二階段學決策樹與隨機森林，理解非線性、交互作用、過擬合和特徵重要度。第三階段依資料型態選修 SVM、KNN、單純貝氏、K-Means 與 PCA。第四階段再進入神經網路、影像、文本或時序模型，並同步學習部署、監控與實驗管理。

真正的進步不是背下更多模型名稱，而是能把模糊需求轉成可驗證問題，建立可信資料切分，選擇符合成本的指標，使用最簡單可行模型完成閉環，再從失敗診斷決定下一步。模型只是決策系統中的一部分；資料、流程、門檻、人工協作與監控共同決定最終價值。

21. 特徵工程實務手冊

21.1 特徵必須對應可重複的資訊

好的特徵不是「和標籤高度相關」而已，而是能在預測時穩定取得、定義一致、形成機制可以說明，且在未來仍可能有效。建立特徵前，先寫出來源資料表、計算時間窗、聚合層級、更新頻率、缺失處理與負責系統。若同一個「近 30 天交易次數」在離線分析採日曆日，上線服務卻採最近 720 小時，結果就可能不同。

每項候選特徵可用四題審查：它在預測當下已經存在嗎；它是否直接或間接包含標籤；它在不同資料來源中的定義是否相同；它變化時能否解釋是行為改變還是資料管線改版。無法回答的欄位不宜直接進入正式模型。

21.2 數值特徵

數值欄位先檢查單位、範圍、偏態與零值意義。收入、金額、流量與等待時間常呈右偏，可比較 $\log_{1p}$ 轉換；比例應確認分母是否為零及不同分母規模造成的可靠度差異。極端值可能是輸入錯誤，也可能是最重要的風險事件，應透過領域規則標記或截尾比較，而不是一律刪除。

聚合特徵應同時保留水準、趨勢與波動。例如設備資料可計算最近一小時平均、標準差、最大值、斜率及超出控制界線的次數；會員資料可計算最近購買日、購買頻率、金額及相對於個人歷史的變化。不同時間窗可能捕捉短期異常與長期基準，但過多高度重複的窗口會增加共線性與維護成本。

21.3 類別特徵

低基數類別可用 one-hot 編碼；高基數類別若直接 one-hot，可能產生巨大稀疏矩陣。可考慮頻率編碼、hashing、目標編碼或 embedding，但目標編碼必須在交叉驗證折內計算，否則會洩漏標籤。新類別一定會出現，前處理必須定義 unknown 行為。

郵遞區號、設備代碼與商品編號看似數字，實際上通常是類別，不應直接當連續值。若代碼本身只代表識別，模型可能記住歷史實體而無法泛化；可改用區域、產品族群、設備世代或歷史統計等有意義的衍生特徵。

21.4 時間特徵

時間不只拆成年、月、星期。應依業務週期加入工作日、假日、班別、結帳週期、產品生命週期與事件前後天數。週期性欄位可用正弦與餘弦表示，避免星期日與星期一在數值上距離很遠。所有滾動統計都要排除當下尚未完成的資料，並處理資料延遲。

若模型會預測未來，驗證也要模擬未來。可以使用 rolling-window 或 expanding-window backtest，逐段以過去訓練、未來驗證。這能看出旺季、政策變動、設備改版或新產品上市造成的穩定性問題。

21.5 文字與影像特徵

文字基準可從字詞 n-gram、TF-IDF 與單純貝氏或線性模型開始。先檢查斷詞、編碼、重複模板、簽名檔與流程代碼；有些高分模型其實只辨識客服模板，而不是內容。進入語言模型前，先定義長度截斷、個資遮蔽、未知詞與推論成本。

影像專案要關注資料拍攝流程。若瑕疵圖都來自某一台相機、某個背景或特定檔名，模型可能學到環境差異。切分資料時應以產品批次、設備或拍攝日期分組。資料增強要模擬真實變化，不能創造不可能的方向、顏色或幾何形狀。

21.6 特徵選擇

特徵選擇的目標是降低噪聲、成本與風險，不是追求欄位越少越好。先移除常數、近常數、明顯重複、預測時不可得及品質不穩定欄位；再使用正則化、permutation importance 或逐步消融比較。選擇程序必須位於交叉驗證內，否則同樣會讓驗證結果過度樂觀。

每次移除特徵都要同時記錄模型指標、推論成本、資料取得成本與解釋性。若移除 30% 欄位只損失極小成績，卻可減少跨系統依賴，通常是更好的正式方案。

22. 實驗設計與可重現交付

22.1 一次實驗只回答一個主要問題

有效實驗應寫成「改變什麼、保持什麼、用什麼資料與指標判斷」。例如「在相同資料切分和前處理下，隨機森林是否比邏輯回歸提高最新月份的 recall@200」，比「嘗試各種模型看看」更容易形成結論。若同時更換資料、特徵、模型與門檻，即使成績提升，也無法知道原因。

建立實驗表時，至少記錄實驗編號、目的、資料版本、切分規則、特徵版本、模型與參數、主要指標、次要指標、資源成本、結果摘要及下一步。失敗實驗也要保存，避免團隊反覆走相同路線。

22.2 交叉驗證不是固定五折

一般獨立同分布資料可使用 KFold 或 StratifiedKFold；同一個人、設備或門市有多筆資料時使用 GroupKFold；有時間先後時使用時間切分。折數越多不一定越好，訓練成本會上升，而且相鄰時間折可能仍高度相似。驗證設計要優先模擬上線時遇到的未知資料。

比較模型時應使用相同切分，報告各折分數與標準差。若平均提升小於折間波動，不宜宣稱新模型穩定優勝。可以進一步觀察每一折的錯誤是否集中在相同族群，判斷問題來自模型還是資料分布。

22.3 誤差分析模板

模型訓練完成後，抽取四組樣本：高信心正確、高信心錯誤、低信心正確、低信心錯誤。對每組檢查資料品質、標籤、特徵可用性與業務情境。高信心錯誤最值得優先調查，因為它可能揭露洩漏、錯誤標籤、罕見族群或模型盲點。

分類問題可依月份、產品、地區、設備、客群、新舊用戶與資料完整度拆解 precision 和 recall；回歸則比較各區間 MAE、偏差方向與極端誤差。不要只列出案例，要將發現轉成可驗證假設，例如增加特徵、修正標籤、調整時間窗或建立獨立模型。

22.4 模型卡應寫什麼

模型卡是給使用者、維運者與稽核者看的短文件，至少包含用途、禁止用途、輸入輸出、訓練資料範圍、主要指標、各族群結果、門檻、已知限制、監控和負責人。若模型只適用特定產品或地區，必須明確寫出，不能期待使用者從程式碼推測。

模型卡也應說明輸出分數的意義。排序分數、未校準機率與校準後風險機率是不同概念。若前端顯示「風險 82%」，團隊必須能證明這個數字在相似樣本中大致對應實際事件比例。

22.5 驗收會議的決策資料

正式上線前，不要只展示一張 AUC 圖。建議同時提供基準與候選模型比較、不同門檻下的警示量、預估人工工時、關鍵族群表現、最常見錯誤、推論成本、監控方案與回滾條件。讓決策者可以在效益、風險與成本之間做取捨。

若仍有重大未知，例如新產品完全沒有歷史資料，應採 shadow mode 或限制適用範圍，而不是用一個整體平均指標掩蓋。清楚承認未知，比部署一個看似完整但無法驗證的模型更專業。

23. 問題排除手冊

23.1 模型幾乎沒有預測能力

先檢查標籤是否正確對齊時間與實體，再確認特徵是否包含與目標有關的資訊。比較簡單統計：正負類特徵分布、目標自相關、不同時間窗與既有規則。如果基準也沒有訊號，問題可能是預測時間太早、標籤噪聲太高或資料根本不包含必要資訊。

接著檢查前處理是否把資料破壞，例如類別全部變成未知、標準化產生 NaN、日期錯位、訓練與驗證欄位順序不同。只有在資料流程確認後，才測試更複雜的特徵和模型。

23.2 訓練很好，驗證很差

優先搜尋資料洩漏與重複樣本，確認同一實體是否跨集合。降低樹深、增加葉節點樣本、加強正則化或縮小神經網路可以減少方差，但若訓練和驗證分布不同，單純正則化不會解決。應比較兩集合的時間、地區、設備與特徵分布。

若資料量少，可減少特徵、使用更簡單模型、做群組或時間交叉驗證，並呈現不確定性。不要反覆修改到驗證集成績變好，否則驗證集也會被過度擬合。

23.3 離線很好，上線後警示暴增

檢查輸入 schema、缺失率、新類別、單位和前處理版本。常見原因是上線資料使用不同時區、累積窗口含當日未完成資料、類別 mapping 遺漏，或某欄位從百分比改成小數。比較離線與線上同一筆樣本的逐步特徵和分數，是最快的定位方式。

也可能是族群比例或基礎事件率改變。此時固定門檻會產生不同警示量，需要重新評估校準與容量。未查明原因前可啟動上限、轉人工或回退舊版本，避免大量錯誤行動。

23.4 模型分數正常，但業務沒有改善

先確認模型輸出是否真的被使用，以及使用者是否在可行時間內收到結果。高風險名單若到事件發生後才送達，模型再準也沒有價值。再檢查行動是否有效：聯繫流失客戶、檢修設備或覆核交易本身可能沒有改善結果。

需要把「模型找到誰」與「對這些人採取行動是否有效」分開。可用隨機對照、A/B test 或分階段推行估計介入效果。預測模型找出高風險，不等於這些對象最容易因介入而改變；必要時進一步研究 uplift 或因果方法。

23.5 特徵重要度與領域直覺衝突

先核對欄位定義、單位與時間，查看是否為代理變數或事後欄位。樹內建重要度可能偏好高基數與可切點多的欄位，應用 permutation importance、消融實驗及局部解釋交叉確認。高度相關特徵會分攤或互相替代重要度，單一欄位排名可能不穩定。

領域直覺也可能不完整。正確做法不是立即刪除衝突特徵，而是提出可檢驗假設，抽查案例並和資料擁有者確認形成機制。若仍無法解釋且風險高，可先限制使用或改用更透明模型。

23.6 模型需要頻繁重訓

頻繁失效可能不是模型問題，而是上游流程、產品、價格或標籤規則持續改變。先用資料監控找出漂移欄位，確認變化是否合理。若欄位本身不穩定，可移除、重新定義或建立更穩健的相對特徵。

重訓頻率應依資料與決策週期設定，並有明確觸發和驗收。每次自動重訓都可能引入新錯誤，因此需要資料品質閘門、候選模型比較、最低指標、模型大小與延遲檢查，以及可回滾版本。

23.7 團隊不知道該繼續調參還是停止

先畫出目前方案相對基準的增益、交叉驗證變異、估計業務價值與額外成本。若多次實驗只在波動範圍內變化，應停止盲目調參，轉向資料、標籤、流程或決策設計。若模型已達成明確成功門檻，也應優先完成上線閉環，從真實使用取得下一輪資訊。

停止條件可以預先寫入專案：達到最低 recall 與最高誤報量；連續若干實驗無顯著改善；推論成本超過限制；或主要剩餘錯誤來自不可取得資訊。明確停止條件能避免模型研究無限延長。